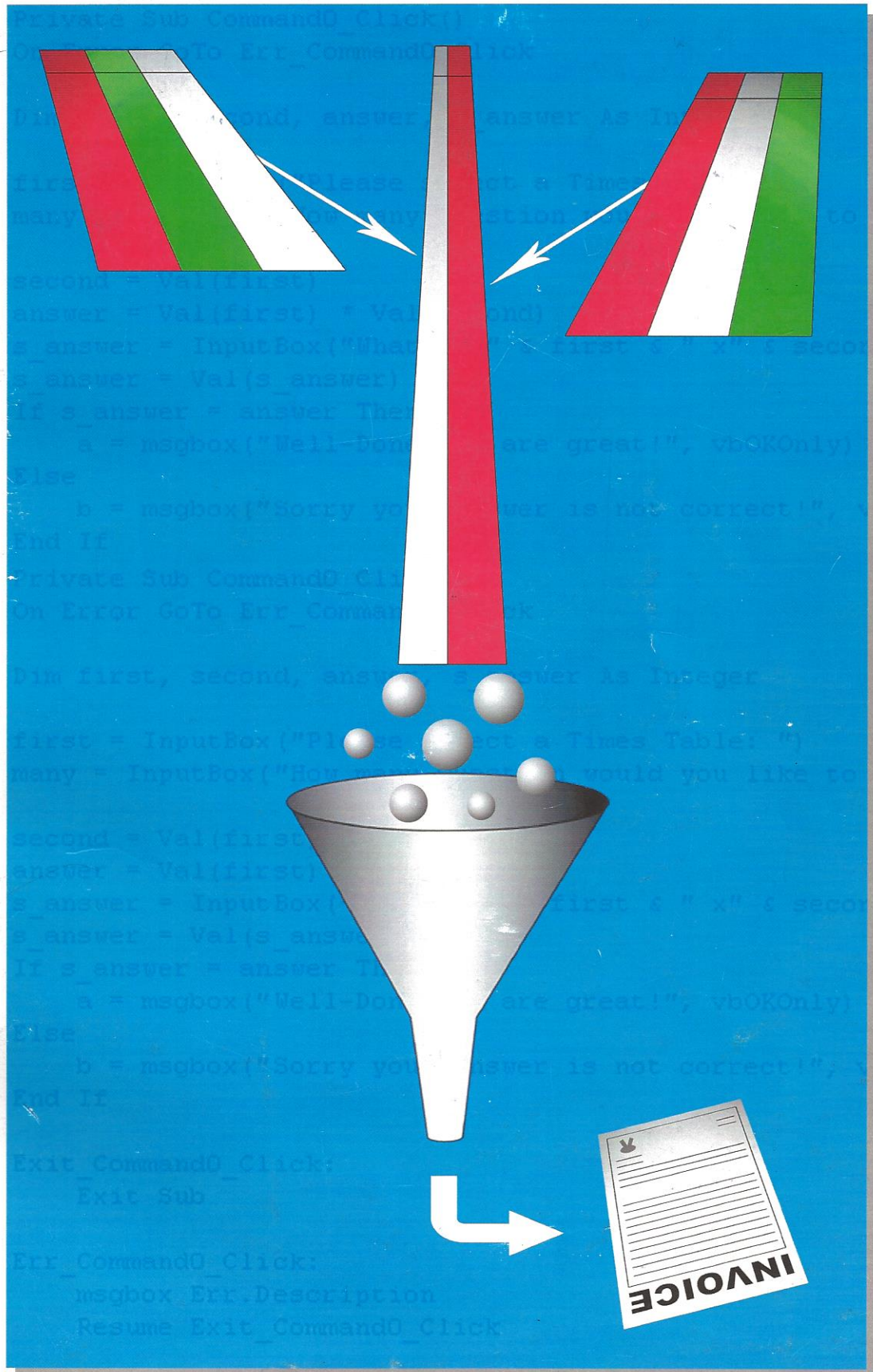


Microsoft **ACCESS** The Easy Way II Database Programming



A step-by-step guide for the novice database student

By Wilhelm Rothman

Copyright © 1998 by Wilhelm Rothman
© WcR 2003

All rights reserved. No part of this book may be reproduced in any form, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the copyright owner.

First Published 1998
Second Edition 2000
Third Edition 2003

ISBN: 1-919746-20-X

® Microsoft Access is a trademark, and Microsoft and Windows are registered trademarks of Microsoft Corporation.

Acknowledgements

The author wishes to acknowledge the contribution made by the following:

- My family, and other friends, who gave me leave of absence.
- My students, who gave me new ideas and inspiration.
- Charles Laurence, a friend, who initially kick started the event.
- Margot for motivation and perseverance.

CHAPTER 11 NORMALIZATION

The aim of normalization is to limit redundancy, in order to manage data optimally. The word optimal is used, because sometimes one might over normalise relations. In extreme cases controlled redundancy must be used in order to have better control over data. Ask the facilitator to explain the term controlled redundancy.

Normalization is a step-by-step process, which consists of a few definitions and rules, in order to limit any unnecessary duplication or redundancy. It forms an integral part of Systems analysis and design, together with the Logical data structure (LDS) (or Entity Relationship-) diagrams and Physical document flow diagrams (PDFD). These methods are ideally executed by three independent groups of analysts, in order to stay objective in their findings. When these models are compared, a study can be done on the differences, e.g. the number of entities identified in an LDS must correlate with the number of relations identified in normalization, which equal the number of data stores in a PDFD eventually.

The comparison might also point out erroneous (redundant) entities identified. The latter of the methods are beyond the scope of this book. Even the normalization within this chapter is covered very briefly, but practice makes perfect and the only way to master the technique is to do as many examples one can find. One must also know and understand every definition, which accompanies the following normal forms.

Normalization can lead to good table structures, which will complement good database design thus eliminating data anomalies caused by the basic file operations (DUI).

Normal Forms:

Zero, First, Second, Third, BCNF, Fourth, Fifth and Domain Key (DKNF) Normal Forms.

I shall only discuss up to BCNF.

Important to note that the anomalies caused by the (D-delete U-update I-insert) basic file operations are triggers, which will start a process of normalization on relations to transform them into a higher normal form. It is important to note that if anomalies do exist, but the relation(s) fulfill the prerequisites of a normal form (definitions and rules), one may consider the relation(s) in that particular normal form. “Cross the bridge when one gets there” attitude should be followed and the relations should be normalised only if the prerequisites are not satisfied. It is true though, if any relation does not meet the prerequisites, usually those relations do have anomalies caused by DUI.

The example of an invoice will be used. It is imperative that one thinks about what processes are involved when an invoice is created, as well as the relationships amongst the entities described by the invoice.



Computer
Promotions Gwd

Order Date: 05 September 2000
Delivery Date: 06 September 2000

Qty	Itemno	Description	Unit Price	Amount	
2	D1T	Ditto 2Gig Tape	140.00	280	00
4	S16R	16 MB Ram Sims	260.00	1,040	00
3	MK12	Mitsumi Keyboards	90.00	270	00
				1,590	00
				222	60
				1,812	60

The first question to be asked, is whether the R_0 table contains any anomalies, when the three basic file operations are performed?

The aim of looking at anomalies is that any table should conform to the integrity rules of which Entity Integrity is a starting point. Entity Integrity says that every table must have a primary key. This primary key, which forms part of an occurrence of such an entity, (row of values or tuple) may not carry null values, but must carry a unique value. Unique in the sense of identifying a row of such an entity in such a way that functional dependency does exist, which answers the question previously put on “What is a primary key?”

THE BASIC FILE OPERATIONS (DUI)

Deletion: Should an order be deleted, data may be lost concerning an item, as well as the client. If the item should be the only item ordered, all references to that item are lost. The reason for this is that if an order is deleted, the deletion removes a whole row.

Updating: Should one change e.g. the item description, every item description must be changed as many times as clients placed orders for that specific item.

Insertion: If an item is inserted (added to the database) a client must exist in order to place an order first.

It thus seems that anomalies do exist for all file operations. The solution in the removal of anomalies is to normalise to the next higher normal form, also known as First Normal Form.

Exactly how you need to go about in rewriting the relation into First Normal Form are illustrated by two techniques. **Firstly** any repeating groups must be identified and the keys of each group can then be concatenated. **Secondly** one can remove the repeating groups by following a few steps. Both techniques will eventually provide the same answer, but the work and energy put into them are different.

One must at first understand what a “repeating group” means. A repeating group can be seen as those set of attributes for which the values change for another attribute or set of attributes where the values remain the same.

In order to determine a repeating group it can be done, by drawing crosses for the occurrences, which are different. I shall illustrate the technique by using the Order example:

R₀(Ord#, Odate, Ddate, Client#, Clientdet, Item#, IDesc, UPrice, Qty, Amount, Total)

```
----- XXXX XXXX XXXXXX XXXX XXXXXX -----
          XXXX XXXX XXXXXX XXXX XXXXXX
          XXXX XXXX XXXXXX XXXX XXXXXX
```

This must now be seen as if one big file exists for **R₀** and the data as indicated by a ----- dotted line will have to be retyped for each x or item detail. Thus for a specific order one may have several items. The crosses (x) indicate the repeating group, as one can see on the invoice.

The aim of the second method is to remove repeating group(s). You may hide the repeating group(s) by covering them with a piece of paper. Only the singleton group will remain of which the Ord# is the primary key. When using the term singleton, one must go back to mathematics and look up the meaning of the term function. A relation now in normalization can only be defined a normalized table, if every non-key attribute (within that table) is determined by a specific value and not more than one value. This specific value is better known as a key or primary key-value. In other words one can say that for a specific key-value there should only be one specific set of values (non-keys) represented by attributes or after mapping your tables to an implementation database model, they are known as (fields) column names. An example would be one's identification number or ID, which should only point to one's First and Last Names, address and other personal particulars known as attributes, which will identify oneself within a database and not somebody else as well. A normalized table can thus be seen as a function. Please obtain the definition of a function, write it down on this page and refer to it when needed to enhance your understanding.

FIRST NORMAL FORM (1NF)

The repeating group can thus be seen as follows, which one should cover (hide) in order to concentrate on the singleton set of attributes.

R₀(.....Item#, IDesc, Uprice, qty, Amount,)

R₁ (**Ord#**, ODate, Ddate, Client#, Cdet, Total)

The next step is to examine the remaining attributes and to hide any other repeating groups or to remove them, and then write down the singleton group, but now the key identified within the singleton group must be concatenated with the key of the parent relation (a typical feature of relational databases, where the primary key matches a foreign key in a subordinate (child) relation. Luckily there are none.

The relation two, R₂ is now created, by concatenating the key of the parent relation with the key of the remaining group, which at this stage forms a singleton group.

R₂(**Ord#**, **Item#**, IDesc, Uprice, qty, Amount)

Currently the keys of both relations will identify or determine occurrences formed by non-key attributes uniquely. Verify the statement by substituting values for the attributes.

By means of a trivial inspection it can be determined that attributes like IDesc, Uprice are determined by Item# only. According to the **principle of minimality** the minimum number of attributes must be determined, necessary to identify an attribute or set of attributes uniquely.

For example examine your ID-Number in the following situation:

In $R_a(\text{ID\#}, \text{Marriage Reg number}, \text{Vehicle Reg Number}, \text{Surname}, \text{name}, \text{address})$ only your ID# is necessary to identify yourself uniquely. Your ID# is thus the minimum number of attributes necessary to identify yourself uniquely as a person.

First normal form therefore carries two prerequisites:

1. The relations must be in their atomic form, which indicates that any non-key attribute (A non-key attribute is an attribute that does not form part of a key) must be determined by a key, in such a way that,
2. Functional Dependency among attributes does exist. Functional Dependency is when non-key attributes are determined by a key, in such a way that for a specific (every) key-value or occurrence of the key-attribute, only one set of values or attribute occurrences do exist. One can therefore not find multi-values for a specific key-value. For example let's look at your ID#. Your ID# may not point to more than one person. If you lookup your ID#, it will only be linked to one set of attribute occurrences.

Once again both relations must be tested against the basic file operations (DUI).

You will notice that R_1 demonstrate the following anomalies.

If you want to change the name of the client, it must be done for each order placed by that client. Should a new client be added to the database, an order must first be placed. Should an order be deleted or cancelled, the whole row will be lost, including the client. If the client only placed one order, the client will also be lost, because it forms part of the row.

The main reason for all the anomalies is that the key must be filled with a value in order to retrieve that row at a later stage.

R_2 also carries data anomalies.

Delete: If an order is deleted, the item will be lost and if the item appears only once, the item is lost forever.

Update: If one wants to change the description of an item, it must be done so for each order, carrying that specific item.

Insert: If one wants to add a new item to the database, an order must first be placed.

SECOND NORMAL FORM (2NF)

One must thus normalize to the next or second normal form.

Prerequisites for 2NF

1. The relations must be in 1NF.
2. Full Functional Dependency must exist.

Every relation is in 1NF and we must only conduct an investigation on Full Functional Dependency or FFD.

Full functional dependency exists when any non-key attribute is dependent on the key and nothing but the key. This is also known as no partial dependency. I shall use the following as an example: $R_b(\underline{A, B}, C, D)$ In this relation C and D are dependent on A&B, but say after a trivial inspection on the data, it is found that C is dependent on A. C is thus partially dependent on A&B and not fully dependent on A&B. One must thus ensure that all non-key attributes are dependent on the whole key and not on part thereof.

In order to solve the problem, one must form subsets of the key, with its non-key attributes, only dependent on that key, e.g. A or B or A,B which will act as keys.

Revisiting our example, you will notice that the key in R_1 only consists out of one attribute. It is therefore impossible that any partial dependency can exist. In this situation all non-key attributes are fully functional dependent on the key. There are anomalies though, but for the sake of the definition, the relation is in 2NF.

$R_1(\underline{\text{Ord\#}}, \text{Odate}, \text{Deldate}, \text{Client\#}, \text{Cdetails}, \text{Total})$

The following relation does have a problem with FFD.

$R_2(\underline{\text{Ord\#}}, \underline{\text{Item\#}}, \text{Idesc}, \text{Uprice}, \text{quantity}, \text{Amount})$

You will notice that Item# determines Idesc and uprice by itself. We now follow the rule in forming subsets of the key.

The following subsets are formed:

Ord#, and Item#, and Ord#, Item#

The first subset is covered by R_1 . Covered in the sense that this is one database we are looking at and will already appear in a table. The other two subsets must be examined. We shall also break up the 2nd relation namely R_2 into 2.1 and 2.2 respectively. The notation will also act as a way to determine the origin of the relation.

$R_{2.1}(\text{Item\#}, \text{Idesc}, \text{Uprice})$

$R_{2.2}(\text{Ord\#}, \text{Item\#}, \text{quantity}, \text{amount})$

Any anomalies concerning $R_{2.1}$ and $R_{2.2}$ are now solved. You can redo the exercise and test all basic operations on the relations. You know however from a previous test R_1 is still a problem. Thus all three relations must be tested for the next normal form.

THIRD NORMAL FORM (3NF)

The prerequisites for Third Normal form or 3NF are:

1. The relations must be in 2NF.
2. Transitive dependency may not be present.
3. Remove any calculated fields or attributes. (like Amount and Total Cost)

All relations are in 2NF, but a study on transitive dependency must be done. Transitive dependency is when a non-key attribute determines another non-key attribute. A Non-key attribute can be seen as any attribute, which does not form part of a key. Such an attribute is a determinant.

Please study the following examples as illustration:

EXAMPLE 1

$R_c(\underline{A}, C, D)$

From the example C and D are determined by the key, because A is the determinant. After inspection of the data you discover that C determines D and C is thus a determinant. We can now say that D is dependent on A via C, or that D is transitively dependent on A. In order to remove transitive dependency one must transform C (determinant) into a key. It is a common fact that transitive dependency will lead to anomalies as discussed earlier.

The solution to $R_c(\underline{A}, C, D)$ is the following:

$R_{c.1}(\underline{A}, C)$ and $R_{c.2}(\underline{C}, D)$

The attributes A, B, C and/or D can refer to a single or set of attributes.

EXAMPLE 2

Given $R_d(\underline{A, B}, C, D)$ In the following case C and D are determined by both A&B, but after a trivial inspection it is found that D determines B. D is thus a determinant. There is no transitive dependency present though, because B forms part of the primary key and is therefore not per definition a non-key attribute. This situation is somewhat different, although the solution to the problem is still a determinant, which must be transformed into a key. Both cases are solved in the same manner, but R_c is not in 3NF or BCNF, but R_d is in 3NF according to the definition, but not in BCNF. If D determines B, then A&D will also determine whatever attributes were determined by A&B. The solution for $R_d(\underline{A, B}, C, D)$ is thus as follows:

$R_{d.1}(\underline{D}, B)$ and $R_{d.2}(\underline{D}, A)$ The attributes A, B, C and/or D may refer to a single or set of attributes. Now a new D-value may be inserted into the database without having to assign values to the A&B key. This should not be seen as the total solution to the problem. Because we are only looking at a snapshot of the database.

BCNF – BOYCE-CODD NORMAL FORM

BCNF or Boyce-Codd Normal form states that all determinants must be transformed into candidate keys. Let us now revisit our Invoicing database.

$R_{1.1}(\underline{\text{Ord\#}}, \text{Odate}, \text{Deldate}, \text{Client\#}, \text{Total})$

$R_{1.2}(\underline{\text{Client\#}}, \text{Cdetail})$

Finally any calculated fields or columns may be omitted in third normal form, because the database can do this type of processing for one. The final answer:

$R_{1.1}(\underline{\text{Ord\#}}, \text{Odate}, \text{Deldate}, \text{Client\#})$

$R_{1.2}(\underline{\text{Client\#}}, \text{Cdetails})$

Cdetail can be broken up into Address1, ... Addressn, Town and Postal Code.

$R_{2.1}(\underline{\text{Item\#}}, \text{Idesc}, \text{Uprice})$

$R_{2.2}(\underline{\text{Ord\#}}, \underline{\text{Item\#}}, \text{quantity})$

IDENTIFICATION OF ALL ENTITIES

Normalization gave one the opportunity to restructure and design good table structures, especially for relational databases. This fact is highlighted by some columnnames being repeated for several tables or relations (not necessary in e.g. Network Database Management Systems – IDMS(x)). This is also seen as necessary duplication.

R_{1,1}(Ord#, Odate, Deldate, Client#)

R_{1,2}(Client#, Cdetail)

Cdetail can be broken up into Address1, ... Addressn, City but for the sake of brevity one can use Cdetail and at the time of implementation it can then be broken down into its sub components. Be careful to include Postal Code, because Postal Code determines City in some cases for example.

R_{2,1}(Item#, IDesc, Uprice)

R_{2,2}(Ord#, Item#, quantity)

Every R or relation representation can be seen as an entity, table or file. One can during the mapping of the database allocate more descriptive names to the entities.

Important is the term connectivity, which describes the classification of a relationship, whether a one to one (1:1), a one to many (1:m) or a many to many (m:n). Ask the facilitator to explain more in detail.

Please complete the following Logical data structure diagram (LDS) on the normalized tables. Indicate the connectivity of the relationships, primary and foreign keys. It is now obvious that our design provided the **R_{2,2}**(Ord#, Item#, quantity) relation or table. *To define the key of R_{2,2} in Microsoft Access during table creation, one must click on Ord#, depress the shift key and move the cursor downwards, in order to highlight Item# as well. While holding the shift key depressed, right-click on the highlighted area in the left-hand margin of the table and select primary key. Both fields will carry a key-icon marker.*

